

B.Sc. Information Technology — Semester VI

Blockchain Technology

Course Code: 25UBIT602

UNIT 4

DApps & Blockchain Tools

Topics Covered:

- ① Decentralized Applications (DApps)
- ② Web3.js and Ether.js Overview
- ③ Interacting with Smart Contracts
- ④ Truffle Suite — Truffle & Ganache
- ⑤ IPFS — InterPlanetary File System
- ⑥ Token Standards — ERC-20 & ERC-721

Exam Starts: 24th March | Credits: 4 | Max Marks: 150

1. Decentralized Applications (DApps)

1.1 What is a DApp?

A Decentralized Application (DApp) is an application whose backend logic runs on a decentralized blockchain network (via Smart Contracts) instead of a centralized server. The frontend looks like a normal website but the backend is powered by blockchain.

Simple Analogy:

- Traditional App (Web2) = Private gym — company owns building, controls your membership, stores your data
- DApp (Web3) = Public park with community-owned vending machine — rules carved in stone (Smart Contracts), no one can kick you out

1.2 DApp Architecture — 4 Layers

Layer	Component	Description
1. Frontend	HTML / React / JavaScript	What users see — looks exactly like a normal website
2. Wallet	MetaMask / Phantom	User's identity and permission to interact with blockchain — replaces Login with Google
3. Backend	Smart Contracts on Blockchain	Business logic lives on blockchain — not on AWS or Google Cloud
4. Storage	IPFS / Arweave	Large files (images, videos, NFTs) stored on decentralized storage networks

1.3 DApp vs Traditional App

Feature	Traditional App (Web2)	DApp (Web3)
Backend	Centralized Server (owned by company)	Distributed Blockchain (owned by nobody)
Identity	Username/Password (owned by app)	Crypto Wallet (owned by you)
Censorship	Company can ban you anytime	Virtually impossible to ban a user
Data Storage	Stored in private databases	Stored on transparent public ledger
Uptime	Fails if company's server goes down	Zero downtime — runs as long as network exists
Trust	Trust the company	Trust the Smart Contract code

1.4 Real-World DApp Categories

Category	Description	Examples
DeFi (Decentralized Finance)	Trade tokens, lend/borrow crypto without a bank	Uniswap, Aave, Compound

Category	Description	Examples
DeSoc (Decentralized Social)	Social media where you own your profile and followers	Lens Protocol, Farcaster
DePIN (Decentralized Physical Infrastructure)	Earn tokens for providing real-world services like WiFi or mapping	Helium (5G), Hivemapper
NFT Marketplaces	Buy/sell digital art and collectibles as NFTs	OpenSea, Blur, Magic Eden
DAOs	Decentralized organizations governed by token holders voting	MakerDAO, Compound DAO

1.5 Challenges of DApps

- Speed — Every transaction must be confirmed by network — slightly slower than centralized server
- Cost — Must pay Gas for every state-changing interaction on blockchain
- Responsibility — No 'Forgot Password' button — lose wallet keys = lose everything forever
- Complexity — Managing Private Keys is still intimidating for most users

2. Web3.js and Ethers.js Overview

2.1 What are Web3.js and Ethers.js?

Web3.js and Ethers.js are JavaScript libraries that allow frontend applications to interact with the Ethereum blockchain — reading data, sending transactions, and calling Smart Contract functions.

2.2 Web3.js

Web3.js was the first major Ethereum JavaScript library. It is a comprehensive, all-in-one toolkit for interacting with Ethereum.

- First major Ethereum library — created alongside the blockchain itself
- All-in-one toolkit — if a feature exists on Ethereum, Web3.js has a function for it
- Larger in size — comprehensive but heavier
- Best for: Complex applications requiring deep control or legacy enterprise projects

2.3 Ethers.js

Ethers.js was built as a smaller, faster, and more secure alternative to Web3.js. It is the industry standard for most modern DApps.

- Lightweight — significantly smaller than Web3.js (~140kb compressed)
- Security-focused — keeps private keys safe with Provider/Signer model
- TypeScript native — excellent auto-complete, prevents bugs before running
- Best for: Modern DApps, mobile-first projects, clean readable code

2.4 Web3.js vs Ethers.js — Comparison

Feature	Web3.js	Ethers.js
First Released	2015 — alongside Ethereum	2016 — built as lighter alternative
Size	Larger — modular but broad	Tiny — ~140kb compressed
API Style	Functional and verbose	Clean and Object-Oriented
Philosophy	Include everything	Keep it lean and secure
License	LGPL-3.0 (Copyleft)	MIT (Very permissive)
Best For	Complex apps, legacy projects, deep control	Modern DApps, mobile-first, clean code
Security	Good	Excellent — separates Provider and Signer

2.5 Provider vs Signer (Ethers.js concept)

Term	Role	Example
Provider	READ-ONLY connection to blockchain — fetches data	Get balance, read contract state
Signer	WRITE connection — signs and sends transactions	Send ETH, call state-changing function

📌 *Ethers.js separates Provider (reading) and Signer (writing) — makes code safer and cleaner*

3. Interacting with Smart Contracts

3.1 What is Needed to Interact?

To interact with a deployed Smart Contract from a frontend application, you need three things:

Requirement	Description	Example
Contract Address	The unique blockchain address where contract is deployed	0x1234...abcd
ABI (Application Binary Interface)	JSON description of all functions and events in the contract	Tells library how to call contract functions
Provider / Signer	Connection to Ethereum network via wallet or RPC	MetaMask (Signer) or Infura (Provider)

3.2 What is ABI?

ABI (Application Binary Interface) is a JSON file generated when a Smart Contract is compiled. It describes all the functions, their parameters, and return types — allowing JavaScript code to call the contract correctly.

- Generated automatically when Solidity code is compiled
- Acts as a bridge between JavaScript frontend and EVM bytecode

- Without ABI — frontend cannot understand how to call contract functions

3.3 Reading from Contract (Free — No Gas)

```
// Using Ethers.js to READ from contract
const provider = new ethers.BrowserProvider(window.ethereum);
const contract = new ethers.Contract(contractAddress, ABI, provider);

// Call a view function — FREE, no MetaMask popup
const balance = await contract.getBalance();
console.log('Balance:', balance.toString());
```

3.4 Writing to Contract (Costs Gas)

```
// Using Ethers.js to WRITE to contract
const signer = await provider.getSigner();
const contract = new ethers.Contract(contractAddress, ABI, signer);

// Call a state-changing function — Costs Gas, MetaMask pops up
const tx = await contract.deposit({ value: ethers.parseEther('0.1') });
await tx.wait(); // Wait for transaction to be confirmed
console.log('Transaction confirmed!');
```

4. Truffle Suite — Truffle & Ganache

4.1 What is Truffle Suite?

Truffle Suite is a development framework for Ethereum Smart Contracts. It provides a complete set of tools for writing, testing, and deploying Smart Contracts on Ethereum.

4.2 Truffle

Truffle is a Smart Contract development framework that provides a development environment, testing framework, and asset pipeline for Ethereum.

Feature	Description
Smart Contract Compilation	Compiles Solidity .sol files into ABI and Bytecode automatically
Automated Testing	Write JavaScript/Solidity tests for your contracts — run with <code>truffle test</code>
Deployment (Migrations)	Deploy contracts to any network using migration scripts
Network Management	Configure multiple networks — local, testnet, mainnet in <code>truffle-config.js</code>
Console	Interactive console to manually call contract functions — <code>truffle console</code>

4.3 Ganache

Ganache is a personal local blockchain for Ethereum development. It runs on your computer and simulates the Ethereum network — allowing you to deploy and test contracts without spending real ETH.

Feature	Description
Local Blockchain	Runs a full Ethereum blockchain on your local machine
Free Test Accounts	Provides 10 pre-funded test accounts with 100 ETH each
Instant Mining	Transactions confirmed instantly — no waiting like real network
GUI + CLI	Available as desktop application (GUI) or command line (<code>ganache-cli</code>)
Zero Cost Testing	Deploy and test contracts without spending real money

4.4 Truffle vs Ganache — Roles

Tool	Role	Analogy
Truffle	Development framework — compile, test, deploy contracts	The chef who prepares and cooks the food
Ganache	Local test blockchain — simulates Ethereum on your machine	The kitchen where chef practices before the restaurant opens

📌 *Workflow: Write Solidity → Compile with Truffle → Deploy to Ganache → Test → Deploy to Mainnet*

5. IPFS — InterPlanetary File System

5.1 What is IPFS?

IPFS (InterPlanetary File System) is a decentralized, peer-to-peer file storage and sharing protocol. Instead of storing files on a central server (like Google Drive or AWS), files are distributed across thousands of nodes worldwide.

- Decentralized storage — no single server stores your files
- Content-addressed — files identified by their content hash (CID), not by location
- Immutable — content hash changes if file is modified — ensures integrity
- Censorship-resistant — no single entity can delete a file stored on IPFS

5.2 How IPFS Works

Step	Process
1. Upload	File is uploaded to IPFS network — broken into chunks
2. Hash Generated	SHA-256 hash of file content = CID (Content Identifier) e.g., QmXyz...
3. Distributed	File chunks distributed across multiple IPFS nodes worldwide
4. Retrieve	Anyone with CID can retrieve the file from nearest node
5. Verify	CID hash verifies file integrity — tampered file = different CID

5.3 IPFS vs Traditional Storage

Feature	Traditional Storage (AWS S3)	IPFS
Address	Location-based URL (where is it?)	Content-based CID (what is it?)
Control	Company can delete/change files	No single entity controls files
Censorship	Files can be taken down	Highly censorship resistant
Cost	Pay monthly subscription	Free for basic use
Use in Blockchain	Centralized — defeats blockchain purpose	Perfect match for decentralized apps

5.4 IPFS in Blockchain / NFTs

- NFT Metadata — Images and metadata of NFTs stored on IPFS — not on expensive blockchain storage
- DApp Assets — Frontend files of DApps hosted on IPFS — fully decentralized
- Documents — Legal contracts, certificates stored immutably on IPFS

📌 *Example: NFT on OpenSea — the token is on Ethereum blockchain, but the actual image file is stored on IPFS*

6. Token Standards — ERC-20 & ERC-721

6.1 What are Token Standards?

Token Standards are official guidelines that define a common set of rules and functions that a Smart Contract must implement to create tokens on Ethereum. Standards ensure all tokens work with the same wallets, exchanges, and DApps.

📌 *ERC = Ethereum Request for Comments — similar to how internet has RFC standards*

6.2 ERC-20 — Fungible Token Standard

ERC-20 is the standard for creating fungible tokens — tokens where every unit is identical and interchangeable. Like currency — one ₹100 note is equal to any other ₹100 note.

Key Characteristics:

- Fungible — Every token is identical and interchangeable with another
- Divisible — Can be split into smaller units (like 0.5 tokens)
- Most common standard — used for cryptocurrencies, governance tokens, utility tokens

ERC-20 Required Functions:

Function	Purpose
totalSupply()	Returns total number of tokens in existence
balanceOf(address)	Returns token balance of a specific address
transfer(to, amount)	Transfer tokens from caller to another address
approve(spender, amount)	Allow another address to spend your tokens
transferFrom(from, to, amount)	Transfer tokens on behalf of another address
allowance(owner, spender)	Returns how many tokens spender is allowed to use

📌 *Real Examples: USDT, USDC, UNI, LINK, SHIB — all ERC-20 tokens*

6.3 ERC-721 — Non-Fungible Token (NFT) Standard

ERC-721 is the standard for creating Non-Fungible Tokens (NFTs) — unique, one-of-a-kind tokens where each token has a unique ID and cannot be exchanged 1:1 with another token.

Key Characteristics:

- Non-Fungible — Every token is unique — Token #1 ≠ Token #2
- Non-Divisible — Cannot split an NFT — you own whole or nothing
- Unique ID — Each token has a unique tokenId that identifies it
- Used for: Digital art, collectibles, game items, certificates, real estate deeds

ERC-721 Required Functions:

Function	Purpose
ownerOf(tokenId)	Returns address of owner of a specific token

Function	Purpose
balanceOf(address)	Returns how many NFTs an address owns
transferFrom(from, to, tokenId)	Transfer a specific NFT to another address
approve(to, tokenId)	Allow another address to transfer a specific NFT
tokenURI(tokenId)	Returns metadata URL of the NFT (image, name, traits)

📌 Real Examples: CryptoPunks, Bored Ape Yacht Club, NBA Top Shot — all ERC-721 NFTs

6.4 ERC-20 vs ERC-721 — Comparison

Feature	ERC-20	ERC-721
Token Type	Fungible — identical and interchangeable	Non-Fungible — unique and one-of-a-kind
Divisible?	Yes — can have 0.5 or 0.001 tokens	No — must own whole token
Each Token Unique?	No — all tokens are identical	Yes — each has unique tokenId
Use Case	Cryptocurrency, DeFi, governance voting	Digital art, collectibles, game items, certificates
Value	Same value per token	Value varies per individual token
Examples	USDT, LINK, UNI, SHIB	CryptoPunks, BAYC, NBA Top Shot
Standard Functions	transfer, approve, allowance	ownerOf, tokenURI, transferFrom

Unit 4 — Quick Revision Summary

Topic	Key Points
DApp	Backend on blockchain Frontend normal website Wallet = identity IPFS = storage
DApp Architecture	Frontend (React) + Wallet (MetaMask) + Smart Contracts + Decentralized Storage
DApp vs Traditional	No censorship No downtime User owns data Trust code not company
DApp Challenges	Slower speed Gas cost No password recovery Complex key management
Web3.js	First Ethereum library All-in-one Larger size Best for complex/legacy apps
Ethers.js	Lightweight (~140kb) Secure TypeScript Provider/Signer model Modern DApps
ABI	JSON description of contract functions Bridge between JS and EVM Auto-generated on compile
Smart Contract Interaction	Need: Contract Address + ABI + Provider/Signer Read=free Write=Gas
Truffle	Development framework Compile + Test + Deploy contracts truffle-config.js
Ganache	Local test blockchain 10 accounts with 100 ETH each Instant mining Free testing
IPFS	Decentralized file storage Content-addressed (CID) Censorship resistant NFT images
IPFS vs AWS	CID not URL No deletion Peer-to-peer Perfect for blockchain apps
ERC-20	Fungible tokens Interchangeable Divisible Crypto/DeFi USDT, LINK
ERC-721	Non-Fungible tokens Unique tokenId Not divisible NFTs CryptoPunks, BAYC
ERC-20 vs ERC-721	ERC-20: Same value, divisible ERC-721: Unique, whole token only

Important Exam Questions

- What is a DApp? Explain its architecture with comparison to traditional apps (7 marks)
- Compare Web3.js and Ethers.js with their features (5 marks)
- What is ABI? How do you interact with a Smart Contract using Ethers.js? (5 marks)
- What is Truffle? What is Ganache? Explain their roles in blockchain development (5 marks)
- What is IPFS? How does it work? Compare with traditional storage (5 marks)
- What is ERC-20? Explain its functions with examples (5 marks)
- What is ERC-721? How is it different from ERC-20? (5 marks)
- Explain token standards in blockchain with ERC-20 and ERC-721 comparison (7 marks)

✅ All the best! Blockchain Unit 4 — DApps & Blockchain Tools — Fully Covered! 🚀